



SWE 4101

Object Oriented Concepts I

Prepared By:
Samia Nawsheen
Junior Lecturer, CSE

Objectives

01 Reference Type vs
Object Type

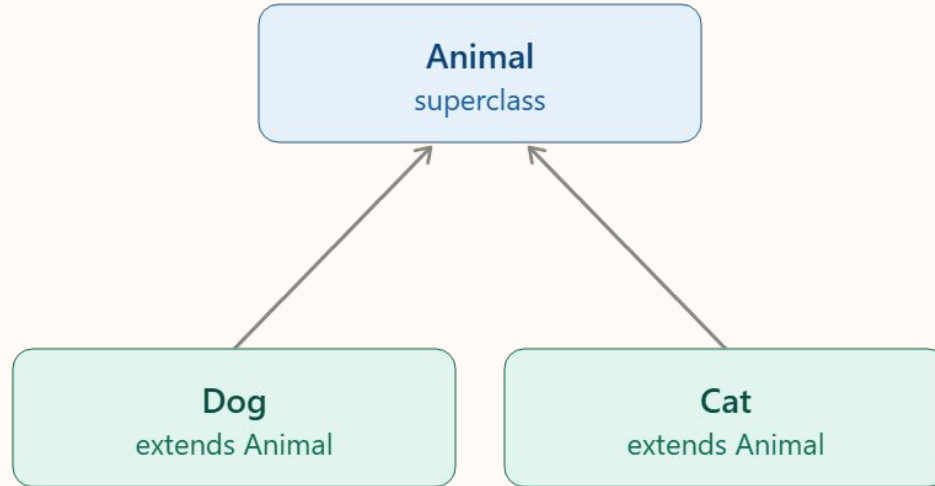
02 Upcasting

03 Downcasting

04 Safe and Unsafe
Casting

05 Instanceof operator

A Refresher on Is-A Relationships



Is every Dog an Animal? Yes

Is every Animal a Dog? No

Object Typecasting

- Object type casting involves **converting** one **object reference type** into **another**
- It applies to objects in an inheritance hierarchy
- Classes must have **Is-A** Relationship
- Two types:
 - Upcasting
 - Downcasting

Upcasting

- Casting **from a subclass to a superclass** is called upcasting
- Always safe
- Done **implicitly** by the compiler
- No need to write cast syntax

Upcasting

```
public class Animal {  
    String type = "Animal";  
    public void eat() {  
        // ...  
    }  
}
```

```
public class Cat extends Animal {  
    String type = "Cat";  
    public void meow() {  
        // ...  
    }  
}
```

```
Cat c = new Cat();
```

```
Animal a = c;
```

```
// Equivalent:
```

```
// Animal a = new Cat();
```

- Implicit upcasting takes place
- There is no need to explicitly cast up the inheritance tree
- Compiler knows that **cat is an Animal**, no error

Attributes Access in Upcasting

What stayed the same?

The object is still a **Cat**

What changed?

The reference is **Animal**

```
System.out.println(animal.type);  
// 'Animal' is printed
```

- Java uses the **reference type** (**Animal**) to determine which field is accessed

Method Access in Upcasting

```
a.eat(); // Compiles
```

```
a.meow(); // Compilation error
```

- Why can't we call meow()?
- The compiler only looks at the **reference type** (**Animal**)
- We can't do anything that is specific to Cat
- We've restricted the number of methods available to **a**

The **reference type** determines what attributes and methods are accessible

Overridden Methods in Upcasting

```
public class Animal {  
  
    void display() {  
        System.out.println("Animal");  
    }  
}
```

```
public class Dog extends Animal {  
  
    @Override  
    void display() {  
        System.out.println("Dog");  
    }  
}
```

```
Animal a = new Dog();
```

```
a.display();  
// 'Dog' is printed
```

- Overridden methods are accessible
- The **object type (Dog)** determines which method runs
- This is runtime polymorphism

Downcasting

```
Animal a = new Cat();
```

- We want to invoke Cat's meow() method on **a**
- But the compiler complains that meow() method doesn't exist for the type Animal
- To call meow() we should convert the reference back to Cat
- We can do this through **Downcasting**

Downcasting

- Casting **from a superclass to a subclass** is downcasting
- Requires explicit casting
- Used to access subclass-specific members
- May throw **ClassCastException** if the object is not actually an instance of the subclass

Safe Downcasting

```
Animal a = new Cat();    // Upcasting  
Cat c = (Cat) a;        // Downcasting  
c.meow();
```

- Why doesn't Java do this implicitly?
- Because Java cannot guarantee that the object is actually a Cat
- This is **safe downcasting**
- Because the object really is a Cat

Unsafe Downcasting

```
Animal a = new Cat();  
Dog d = (Dog) a;
```

- Will this compile?
 - Yes
- Will this run?
 - No
- Java throws **ClassCastException**
- At compile-time, Java assumes **a** is a Dog but at runtime, it sees the actual object is a Cat

Unsafe Downcasting

```
Dog dog = new Animal(); // Compilation Error
```

- Trying to assign an Animal object directly to a Dog reference
- The compiler knows that a general Animal is **not guaranteed** to be a Dog, so it rejects the assignment immediately

```
Animal animal = new Animal();  
Dog dog = (Dog) animal;
```

- This compiles but at runtime, Java checks the object and finds that it's actually an Animal, not a Dog
- Throws a **ClassCastException**

instanceof Operator

- Check whether an object is an instance of a specific class
- Returns either true or false

```
if(animal instanceof Dog)
```

- Checks the actual runtime type

Safe Downcasting with instanceof

```
if (animal instanceof Dog){  
    Dog d = (Dog) animal;  
    d.bark();  
} else if (animal instanceof Cat) {  
    Cat c = (Cat) animal;  
    c.meow();  
}
```

- bark() is called only when animal is actually a Dog
- meow() is called when it is a Cat
- No chance of `ClassCastException`

Safe Downcasting with instanceof

- It's tedious to write this type of code where we need to test the type and make a cast for every conditional block
- Readability is poor as the casting and variable extraction dominate the code
- Repeatedly declaring the type name means there's more likelihood of introducing an error

instanceof Pattern Matching

- Java 14+ allows this:

```
if (animal instanceof Dog d) {  
    // The variable 'd' is automatically cast and ready to use  
    d.bark();  
}
```

- **d** is called a pattern variable here and its scope is within this block only

Rules of Pattern Matching

1. Conditional AND (&&)

You can use the pattern variable immediately on the right side of an && operator because the right side only runs if the type check passes

```
if (animal instanceof Dog d && d.getAge() > 5) {  
    d.bark();  
}
```

Rules of Pattern Matching

2. Conditional OR (||)

You cannot use the pattern variable on the right side of an || operator.
If the left side fails, the variable is not initialized.

```
if (animal instanceof Dog d || d.getAge() > 5) {  
    d.bark();  
}  
  
// Compilation Error
```

Rules of Pattern Matching

3. Negated Condition (!)

If you invert the condition using a negative check, the scope of the variable shifts into the else block

```
public void examineAnimal (Animal animal) {  
    if (!(animal instanceof Dog d)) {  
        return; // d is not in scope here  
    }  
    // d is in scope here  
    d.bark();  
}
```

**Thank
You**